Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Степанова А.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 22.10.26

Москва, 2026

Постановка задачи

Вариант 19.

Дан массив координат (x, y). Пользователь вводит число кластеров. Проведите кластеризацию методом k-средних.

Необходимо составить программу на языке Си, обрабатывающую данные в многопоточном режиме с использованием стандартных средств создания потоков ОС (POSIX Threads). Максимальное количество потоков, работающих в один момент времени, задается ключом запуска программы. Необходимо реализовать последовательную и параллельную версии алгоритма, замерить время их работы и исследовать зависимость ускорения и эффективности от количества потоков и объема входных данных.

Из стандартной библиотеки допускается использование только POSIX Threads (Linux).

Общий метод и алгоритм решения

В лабораторной работе использованы следующие системные вызовы и функции POSIX:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg); – создает новый поток, выполняющий функцию start.
- int pthread_join(pthread_t thread, void **retval); – ожидает завершения потока.
- int pthread_barrier_init(pthread_barrier_t *b, const pthread_barrierattr_t *attr, unsigned count); – создает барьер для count потоков.
- int pthread_barrier_wait(pthread_barrier_t *b); – блокирует поток, пока все count потоков не дойдут до барьера.
- int pthread_barrier_destroy(pthread_barrier_t *b); – освобождает барьер.
- int clock_gettime(clockid_t clk, struct timespec *ts); – получает значение монотонных часов (CLOCK_MONOTONIC) для замера времени.
- unsigned sleep(unsigned seconds); – приостанавливает поток (используется только в режиме демонстрации количества потоков).
- ssize_t write(int fd, const void *buf, size_t count); – вывод сообщений об ошибках в stderr.

Метод k-средних разбивает N точек на K кластеров. В начале в качестве центроидов выбираются первые K точек. Затем повторяются два шага: (1) назначение – каждая точка относится к кластеру с ближайшим центроидом (по квадрату евклидова расстояния); (2) пересчет – центроид каждого кластера заменяется средним арифметическим координат его точек. Алгоритм завершается, когда ни одна точка не сменила кластер, либо после 100 итераций.

Входные данные генерируются программой: N точек с целыми координатами вокруг восьми случайных центров, для чего используется собственный генератор псевдослучайных чисел (xorshift), так как функции стандартной библиотеки использовать нельзя. Данные хранятся в статических массивах. Целые координаты выбраны намеренно: суммы координат вычисляются точно независимо от порядка сложения, поэтому последовательная и параллельная версии дают в точности одинаковые центроиды, что программа проверяет и выводит в строке results match.

Последовательная версия (kmeans_sequential) выполняет оба шага в одном потоке. Параллельная версия (kmeans_parallel) создает заданное ключом запуска число рабочих потоков, один раз в начале работы. Массив точек делится на равные непрерывные части, каждый поток обрабатывает свою часть: для каждой точки определяет ближайший центроид и накапливает в

собственной строке таблицы частичные суммы координат и счетчики точек по кластерам. Так как каждый поток пишет только в свои данные, мьютексы не требуются.

После завершения назначения все потоки ожидают на барьере (pthread_barrier_wait). Затем поток 0 складывает частичные суммы всех потоков, пересчитывает центроиды и решает, продолжать ли итерации. Второй барьер гарантирует, что остальные потоки начнут новую итерацию только после обновления центроидов. Таким образом, на каждую итерацию приходится два барьера. Основной поток только создает рабочие потоки и ожидает их завершения через pthread_join.

Запуск программы: ./main <число потоков> <число точек> <число кластеров> [пауза в секундах]. Необязательный четвертый параметр останавливает поток 0 на заданное время в параллельной части, чтобы можно было успеть посмотреть число потоков процесса стандартными средствами ОС (ps -o nlwp, top -H). Время измеряется функцией clock_gettime для последовательной и параллельной версии отдельно (генерация данных в замер не входит).

Для оценки параллельной версии используются две метрики.

Ускорение (speedup) показывает, во сколько раз параллельная версия быстрее последовательной: $S(n) = T1 / Tn$, где T1 – время выполнения последовательной версии, Tn – время выполнения параллельной версии на n потоках, n – число потоков. Значение $S = 1$ означает отсутствие выигрыша, идеальное (линейное) ускорение равно n.

Эффективность (efficiency) показывает, насколько хорошо используются потоки: $E(n) = S(n) / n = T1 / (n \cdot Tn)$, где S(n) – ускорение, n – число потоков. Значение $E = 1$ (100%) соответствует идеальному использованию всех потоков; чем меньше E, тем больше времени потоки тратят на ожидание, синхронизацию и накладные расходы.

Эксперименты проводились на машине с 16 логическими ядрами (WSL, Linux). Для 2 000 000 точек каждая конфигурация запускалась три раза, в таблицу вносилось наименьшее из трех времен (так минимизируется влияние фоновой нагрузки системы); в качестве T1 взято наименьшее из 27 замеров последовательной версии, 1430.1 мс. Для 1000 точек выполнен один запуск, T1 = 0.092 мс (медиана девяти замеров). Число кластеров во всех экспериментах равно 8.

Код программы

main.c

```
#include <stdint.h>
#include <stdbool.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdio.h>

#define MAX_POINTS  4000000
#define MAX_K       32
#define MAX_THREADS 1024
#define MAX_ITER    100

static double px[MAX_POINTS];
static double py[MAX_POINTS];
static int labels[MAX_POINTS];

static int n_points;
static int n_clusters;

static double cx[MAX_K];
static double cy[MAX_K];
```

```
static double sum_x[MAX_THREADS][MAX_K];
static double sum_y[MAX_THREADS][MAX_K];
static int cnt[MAX_THREADS][MAX_K];
static int changed[MAX_THREADS];

static int iterations_done;
static uint32_t rng_state = 12345;

static uint32_t rng_next(void) {
	rng_state ^= rng_state << 13;
	rng_state ^= rng_state >> 17;
	rng_state ^= rng_state << 5;
	return rng_state;
}

static double now_ms(void) {
	struct timespec ts;
	clock_gettime(CLOCK_MONOTONIC, &ts);
	return ts.tv_sec * 1000.0 + ts.tv_nsec / 1000000.0;
}

static int parse_int(const char *s) {
	int r = 0;
	if (s[0] == '\0')
		return -1;
	for (size_t i = 0; s[i] != '\0'; ++i) {
		if (s[i] < '0' || s[i] > '9')
			return -1;
		r = r * 10 + (s[i] - '0');
	}
	return r;
}

static void generate_points(void) {
	double bx[8], by[8];
	for (int i = 0; i < 8; ++i) {
		bx[i] = rng_next() % 10000;
		by[i] = rng_next() % 10000;
	}
	for (int i = 0; i < n_points; ++i) {
		int b = rng_next() % 8;
		px[i] = bx[b] + (int)(rng_next() % 3000) - 1500;
		py[i] = by[b] + (int)(rng_next() % 3000) - 1500;
		labels[i] = -1;
	}
}

static void init_centroids(void) {
	for (int k = 0; k < n_clusters; ++k) {
		cx[k] = px[k];
		cy[k] = py[k];
	}
	for (int i = 0; i < n_points; ++i)
		labels[i] = -1;
}

static void assign_range(int t, int lo, int hi) {
	for (int k = 0; k < n_clusters; ++k) {
		sum_x[t][k] = 0;
		sum_y[t][k] = 0;
		cnt[t][k] = 0;
	}
	int ch = 0;
	for (int i = lo; i < hi; ++i) {
		int best = 0;
```

```
                double best_d = 1e300;
                for (int k = 0; k < n_clusters; ++k) {
                        double dx = px[i] - cx[k];
                        double dy = py[i] - cy[k];
                        double d = dx * dx + dy * dy;
                        if (d < best_d) {
                                best_d = d;
                                best = k;
                        }
                }
                if (labels[i] != best) {
                        labels[i] = best;
                        ++ch;
                }
                sum_x[t][best] += px[i];
                sum_y[t][best] += py[i];
                ++cnt[t][best];
        }
        changed[t] = ch;
}

static bool update_centroids(int slots) {
        int total_changed = 0;
        for (int t = 0; t < slots; ++t)
                total_changed += changed[t];
        for (int k = 0; k < n_clusters; ++k) {
                double sx = 0, sy = 0;
                int c = 0;
                for (int t = 0; t < slots; ++t) {
                        sx += sum_x[t][k];
                        sy += sum_y[t][k];
                        c += cnt[t][k];
                }
                if (c > 0) {
                        cx[k] = sx / c;
                        cy[k] = sy / c;
                }
        }
        return total_changed > 0;
}

static void kmeans_sequential(void) {
        init_centroids();
        iterations_done = 0;
        while (iterations_done < MAX_ITER) {
                assign_range(0, 0, n_points);
                ++iterations_done;
                if (!update_centroids(1))
                        break;
        }
}

static pthread_barrier_t barrier;
static int thread_count;
static volatile bool stop_flag;
static int pause_sec;

typedef struct {
        int id;
        int lo;
        int hi;
} worker_arg;

static void *worker(void *p) {
        worker_arg *a = (worker_arg *)p;
```

```
	if (a->id == 0 && pause_sec > 0)
		sleep(pause_sec);
	while (1) {
		assign_range(a->id, a->lo, a->hi);
		pthread_barrier_wait(&barrier);
		if (a->id == 0) {
			++iterations_done;
			bool go = update_centroids(thread_count);
			stop_flag = !go || iterations_done >= MAX_ITER;
		}
		pthread_barrier_wait(&barrier);
		if (stop_flag)
			break;
	}
	return NULL;
}

static bool kmeans_parallel(int threads) {
	init_centroids();
	iterations_done = 0;
	stop_flag = false;
	thread_count = threads;
	static pthread_t tid[MAX_THREADS];
	static worker_arg args[MAX_THREADS];
	if (pthread_barrier_init(&barrier, NULL, threads) != 0)
		return false;
	int chunk = n_points / threads;
	int rest = n_points % threads;
	int pos = 0;
	for (int t = 0; t < threads; ++t) {
		args[t].id = t;
		args[t].lo = pos;
		pos += chunk + (t < rest ? 1 : 0);
		args[t].hi = pos;
		if (pthread_create(&tid[t], NULL, worker, &args[t]) != 0) {
			const char msg[] = "error: failed to create thread\n";
			write(STDERR_FILENO, msg, sizeof(msg) - 1);
			return false;
		}
	}
	for (int t = 0; t < threads; ++t)
		pthread_join(tid[t], NULL);
	pthread_barrier_destroy(&barrier);
	return true;
}

int main(int argc, char **argv) {
	if (argc < 4) {
		printf("usage: %s threads points clusters [pause_sec]\n", argv[0]);
		return 0;
	}
	int threads = parse_int(argv[1]);
	n_points = parse_int(argv[2]);
	n_clusters = parse_int(argv[3]);
	pause_sec = argc > 4 ? parse_int(argv[4]) : 0;
	if (threads < 1 || threads > MAX_THREADS || n_points < 1 || n_points >
MAX_POINTS ||
	    n_clusters < 1 || n_clusters > MAX_K || n_clusters > n_points || pause_sec <
0) {
		const char msg[] = "error: bad arguments\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		return 1;
	}
	generate_points();
	double t0 = now_ms();
```

```
        kmeans_sequential();
        double t_seq = now_ms() - t0;
        int seq_iters = iterations_done;
        double seq_cx[MAX_K], seq_cy[MAX_K];
        for (int k = 0; k < n_clusters; ++k) {
              seq_cx[k] = cx[k];
              seq_cy[k] = cy[k];
        }
        t0 = now_ms();
        if (!kmeans_parallel(threads))
              return 1;
        double t_par = now_ms() - t0;
        bool same = (seq_iters == iterations_done);
        for (int k = 0; k < n_clusters; ++k) {
              if (seq_cx[k] != cx[k] || seq_cy[k] != cy[k])
                    same = false;
        }
        double speedup = t_seq / t_par;
        double efficiency = speedup / threads;
        printf("points=%d clusters=%d threads=%d iterations=%d\n",
               n_points, n_clusters, threads, iterations_done);
        printf("sequential: %.3f ms\n", t_seq);
        printf("parallel:   %.3f ms\n", t_par);
        printf("speedup:    %.3f\n", speedup);
        printf("efficiency: %.3f\n", efficiency);
        printf("results %s\n", same ? "match" : "DIFFER");
        for (int k = 0; k < n_clusters; ++k)
              printf("centroid %d: (%.2f, %.2f)\n", k, cx[k], cy[k]);
        return 0;
}
```

Протокол работы программы

Результаты для 2 000 000 точек:

Запуск: ./main <потоки> 2000000 8. Для каждого числа потоков программа выводит время последовательной и параллельной версий, ускорение, эффективность и результат сравнения центроидов (во всех запусках results match). В таблице приведено лучшее из трех запусков. В первом запуске серии (1 поток) время обеих версий оказалось примерно вдвое больше обычного (2725–2821 мс вместо 1439 мс) – вероятно, из-за состояния системы в момент старта; наименьшее значение исключает этот выброс.

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	1439,3	0,99	0,99
2	748,6	1,91	0,96
4	398,5	3,59	0,90
8	235,7	6,07	0,76
12	242,5	5,90	0,49
16	217,4	6,58	0,41
64	314,7	4,54	0,07
256	689,6	2,07	0,01
1024	2629,2	0,54	0,00

Ускорение и эффективность рассчитаны по формулам S = T1 / Tn и E = S / n при T1 = 1430,1 мс (лучшее из трех запусков).

Результаты для 1000 точек (малый объем данных):

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	0,240	0,3833	0,3833
2	0,699	0,1316	0,0658
4	1,538	0,0598	0,0150
8	2,778	0,0331	0,0041
12	4,582	0,0201	0,0017
16	5,917	0,0155	0,0010
64	22,410	0,0041	0,0001
256	86,502	0,0011	0,0000
1024	365,377	0,0003	0,0000

T1 = 0.092 мс.

Количество потоков процесса:

Для демонстрации количества потоков программа запускается с паузой в параллельной части (четвертый параметр – 20 секунд), и через 3 секунды выполняется ps -o nlwp,cmd. Колонка NLWP показывает число потоков процесса: 17 (16 рабочих потоков и основной поток).

```
nastya@DESKTOP-HOT4H2I:~/projects/os/lab2$ ./main 16 2000000 8 20 &
sleep 3; ps -o nlwp,cmd -p $!
[1] 7553
NLWP CMD
  17 ./main 16 2000000 8 20
```

Рисунок 1 – Количество потоков процесса (ps -o nlwp)

Вывод

В ходе работы реализован алгоритм k-средних в последовательном и параллельном вариантах на POSIX Threads; число потоков задается ключом запуска, количество потоков процесса подтверждено командой ps (NLWP = 17 для 16 рабочих потоков). Результаты обеих версий совпадают во всех запусках. На 2 000 000 точек ускорение до 8 потоков растет почти линейно (1,91 на 2 потоках, 3,59 на 4, 6,07 на 8 при эффективности 0,76), после чего рост практически прекращается: на 12 потоках 5,90, на 16 потоках (число логических ядер) 6,58 при эффективности 0,41.

Ускорение меньше линейного по нескольким причинам. Во-первых, на каждой итерации потоки дважды ожидают на барьере, а суммирование частичных результатов и пересчет центроидов выполняет один поток; эта последовательная часть ограничивает ускорение (закон Амдала). Во-вторых, 2 000 000 точек занимают около 40 МБ и не помещаются в кэш, поэтому на каждой итерации данные читаются из оперативной памяти, пропускная способность которой общая для всех ядер. В-третьих, 16 логических ядер, вероятно, соответствуют 8 физическим ядрам с гиперпоточностью: два логических ядра делят один набор вычислительных блоков, поэтому переход с 8 до 16 потоков дает прирост всего с 6,07 до 6,58.

При числе потоков больше числа ядер ускорение падает: 4,54 на 64 потоках, 2,07 на 256 и 0,54 на 1024, то есть при 1024 потоках параллельная версия медленнее последовательной. Потоков больше, чем ядер, поэтому планировщик ОС переключает их между собой, а барьер заставляет все потоки ждать самый медленный; кроме того, создание и синхронизация сотен потоков требуют времени и памяти. Эффективность при этом стремится к нулю (0,001 на 1024 потоках).

Зависимость от объема данных: на 1000 точках последовательная версия работает 0.092 мс, а параллельная версия медленнее при любом числе потоков (от 0,240 мс на 1 потоке до 365,4 мс на 1024 потоках). Полезной работы так мало, что затраты на создание потоков и синхронизацию на барьерах многократно ее превышают. Поэтому распараллеливание оправдано только для достаточно больших объемов данных, а число потоков следует выбирать не больше числа логических ядер (в данной работе оптимально 8–16 потоков).